Practicum No.2 **Date:…….**

Develop solutions that perform logic operations based on ALU Status Flags

I. Practical Significance

Modern processors use ALU status flags to summarize the outcome of arithmetic and logical operations. Instead of re-evaluating numeric results, CPUs apply logic operations on these flags to decide whether a result is valid, erroneous, safe for further use, or requires special handling. This experiment helps students understand how logic operations on status flags drive processor decision-making, which is a fundamental concept in computer organization, embedded systems, and control-unit design.

II. Competency and Industry-Specific Practical Skills

Use the principles of computer architecture to optimize system performance

a) ***Construct*** logical expressions using Boolean operators. (PDO)
b) ***Implement*** logic-based solutions using MATLAB logical operators. (PDO)
c) ***Interpret*** logical outputs derived from processor status data. (CDO)

III. Relevant CO(s) *(As stated in the Course Structure)*

- **CO1.** ***Use*** the relevant instruction cycle to efficiently manage the processor's functionality.

IV. Practical Outcome (PrO) *(As stated in the Course Structure)*

- ***Develop*** solutions that perform logical operations for ***the given*** ALU status flags.

V. Related ADO(s) *(As stated in the Course Structure)*

a) ***Follow*** ethical practices in designing or evaluating the system design.
b) ***Apply*** systematic design principles and debugging techniques.
c) ***Function*** effectively as a team member.

VI. Minimum Underpinning Theory *(Include relevant content & images for this practicum.)*

Each ALU operation updates a set of status flags such as Carry Flag (CF), Overflow Flag (OF), Sign Flag (SF), and Parity Flag (PF). These flags provide compact information about the operation result. The CPU applies logic operations on these flags using basic gates such as AND, NOT, and NAND to make decisions related to result validity, error detection, and signed interpretation. This hardware-level decision approach ensures fast execution and minimal overhead in processor control logic. Such logic-based interpretation is widely used in processor control units and exception handling.

Flag	0	1
SF (Sign Flag)	Positive result	Negative result
OF (Overflow Flag)	No signed overflow	Signed overflow
CF (Carry Flag)	No carry generated	Carry generated
PF (Parity Flag)	Odd parity	Even parity

VII. Practicum Set-up/Circuit Diagram/Algorithm/Flowchart *(Include probable photos of experimental set-up.)*

Algorithm

1. Perform an arithmetic operation using signed 8-bit data.
2. Record the ALU status flags (CF, OF, SF, PF).
3. Apply logic expressions for the given Scenario.
4. Observe logic output and classify the result.
5. Validate decisions using multiple test cases.

VIII. Resources Required

S. No.	Resource	Suggested Broad Specification	Quantity
1	MATLAB	(R2016b or newer) or GNU Octave (with mostly compatible syntax).	1 No.
2	PC	PC with MATLAB installed.	1 No.
3	Text editor / MATLAB Editor	-	1 No.

IX. Safety Precautions

a) ***Save*** work frequently.
b) ***Comment*** code and include header describing author, date, input format and usage.
c) For shared code, ***remove*** hard-coded absolute paths.
d) ***Validate*** user inputs to avoid invalid logical values and runtime errors.

X. Procedure *(Self-Instructional)*

1. ***Select*** signed input values.
2. ***Perform*** arithmetic and note status flags.
3. ***Implement*** the Scenario using gates.
4. ***Record*** logic output and CPU decision.

Scenario: Safe and Clean ALU Output using AND (**AND gate output should be 1 – for safe and clean**)
Condition after arithmetic operation:
CF = 0, OF = 0, PF = 1
Logic Expression:
SAFE_OUTPUT = $C\overline{F}$ AND $O\overline{F}$ AND PF
CPU Interpretation:
No carry, no signed overflow, and even parity (to avoid error) — the CPU treats the result as safe and clean.

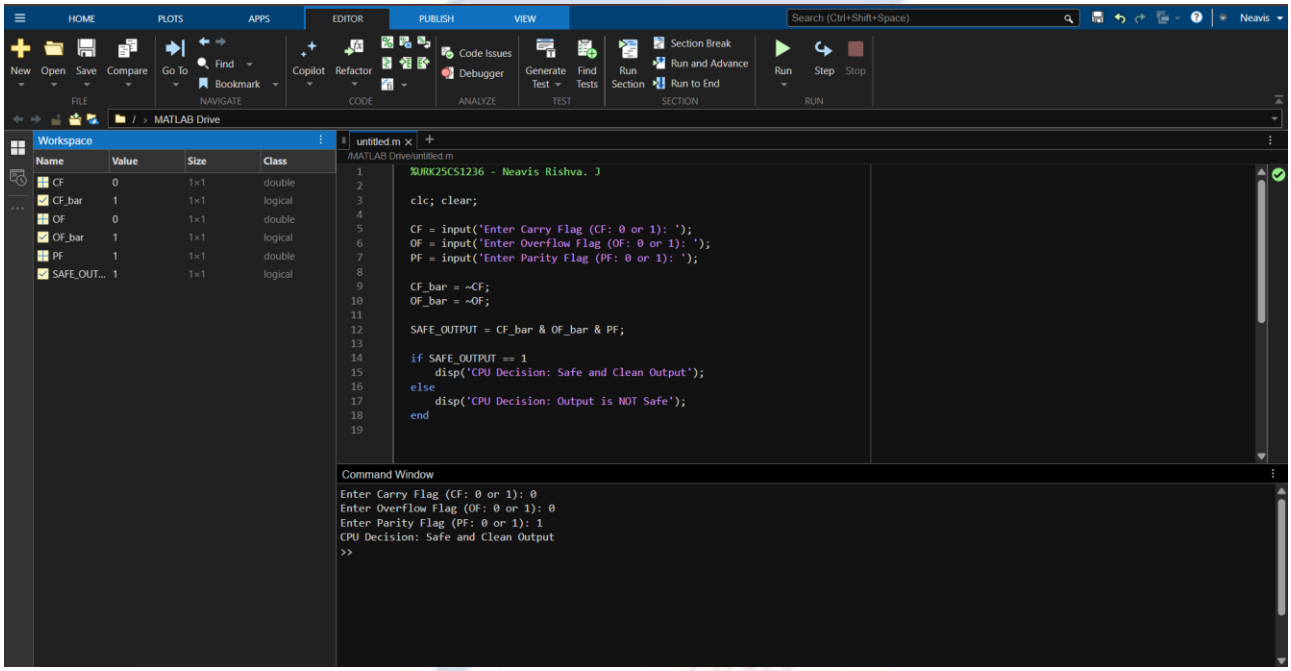
% Scenario: Safe and Clean Output using **AND and NOT gates**

```
clc; clear;

% Input ALU flags (0 or 1)
CF = input('Enter Carry Flag (CF: 0 or 1): ');
OF = input('Enter Overflow Flag (OF: 0 or 1): ');
PF = input('Enter Parity Flag (PF: 0 or 1): ');

% NOT operations
CF_bar = ~CF;
OF_bar = ~OF;

% AND operation
SAFE_OUTPUT = CF_bar & OF_bar & PF;

% Display result
if SAFE_OUTPUT == 1
   disp('CPU Decision: Safe and Clean Output');
else
```

```
    disp('CPU Decision: Output is NOT Safe');
end
```

Sample Input & Sample Output:

Input:
Enter Carry Flag (CF: 0 or 1): 0
Enter Overflow Flag (OF: 0 or 1): 0
Enter Parity Flag (PF: 0 or 1): 1
Output:
CPU Decision: Safe and Clean Output



XI. Actual Resources Used

S. No.	Name of Resource	Broad Specifications		Quantity	Remarks (If any)
		Make	Details		
1.	PC / Laptop	Asus	Intel Core i5 Processor, 8 GB RAM	1	Used for program execution
2.	MATLAB Software	MathWorks	MATLAB R2019a (64-bit)	1	Arithmetic computation
3.	Operating System	Microsoft	Windows 11 (64-bit)	1	Platform support
4.	MATLAB Editor	MathWorks	Built-in Script Editor	1	Code development

XII. Actual Procedure Followed

1. MATLAB was used to implement logic operations using ALU status flags.
2. Arithmetic operations were performed on signed inputs and flags (CF, OF, SF, PF) were identified.
3. Logical gates (NOT, AND, OR, NAND, NOR) were implemented using MATLAB operators.

4. CPU decisions were displayed and verified using multiple test cases, and results were recorded.

XIII. Additional Questions

1. Implement a Valid Negative Result Detection using **NAND** Gate (**NAND output = 0 – Valid**) - VALID_NEGATIVE = NAND (OF̄, SF). **Note:** SF = 1 → Result is negative, SF = 0 → Result is positive.

CODE:

```
clc; clear;

OF = input('Enter Overflow Flag (OF: 0 or 1): ');
SF = input('Enter Sign Flag (SF: 0 or 1): ');

OF_bar = ~OF;

VALID_NEGATIVE = ~(OF_bar & SF);

if VALID_NEGATIVE == 0
   disp('CPU Decision: Valid Negative Result');
else
   disp('CPU Decision: Invalid or Abnormal Result');
end
```

Sample Input & Sample Output:

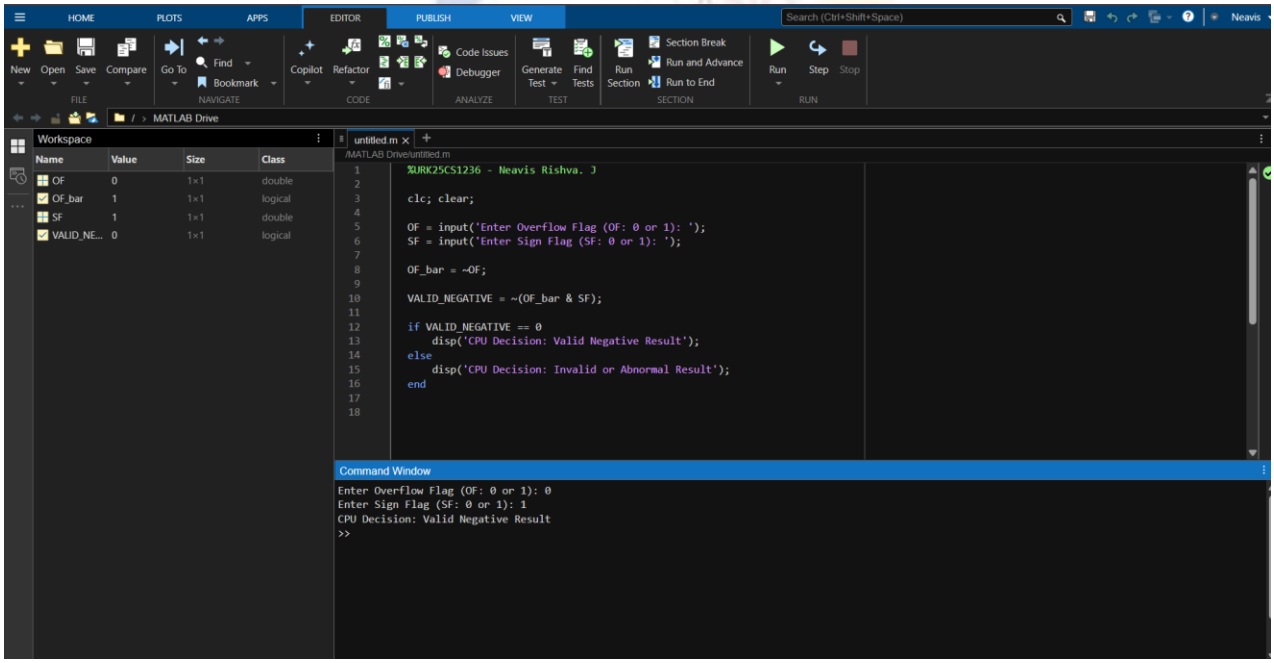
Input:

Enter Overflow Flag (OF: 0 or 1): 0

Enter Sign Flag (SF: 0 or 1): 1

Output:

CPU Decision: Valid Negative Result



2. Implement an Abnormal Result Detection using **OR** Gate (O**R output = 1**) - ABNORMAL = CF OR OF. **Note**: Only when both CF and OF are 0 is the result normal i.e. 0.

Code:

```
clc; clear;


CF = input('Enter Carry Flag (CF: 0 or 1): ');
OF = input('Enter Overflow Flag (OF: 0 or 1): ');

ABNORMAL = CF | OF;

if ABNORMAL == 1
   disp('CPU Decision: Abnormal Result Detected');
else
   disp('CPU Decision: Normal Result');
end
```

Sample Input & Sample Output:

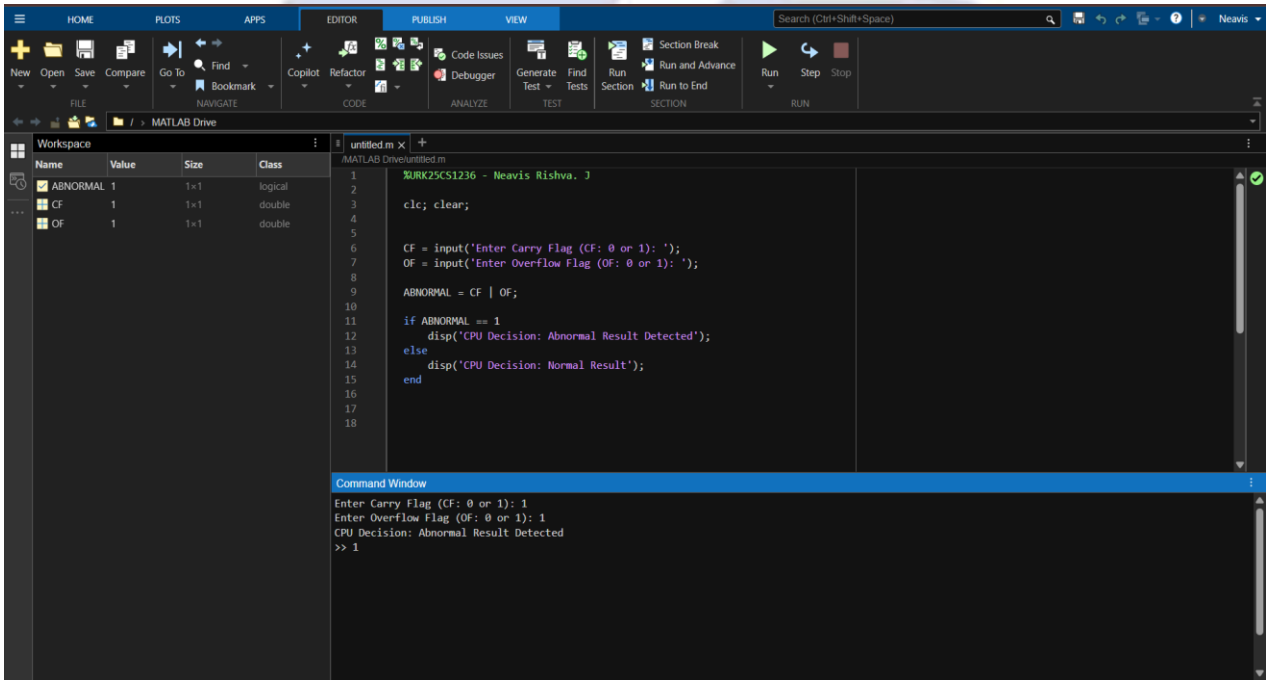
Input:

Enter Carry Flag (CF: 0 or 1): 1

Enter Overflow Flag (OF: 0 or 1): 1

Output:

CPU Decision: Abnormal Result Detected



3. Implement a Clean Result Detection using **NOR** Gate (**NOR output = 1**)- CLEAN = NOR (CF, OF). **Note:** When neither carry nor overflow is detected, the CPU confirms a clean arithmetic result and output of NOR will be 1.

Code:

```
clc; clear;

CF = input('Enter Carry Flag (CF: 0 or 1): ');
```

```
OF = input('Enter Overflow Flag (OF: 0 or 1): ');

CLEAN = ~(CF | OF);

if CLEAN == 1
   disp('CPU Decision: Clean Arithmetic Result');
else
   disp('CPU Decision: Result is NOT Clean');
end
```

Sample Input & Sample Output:

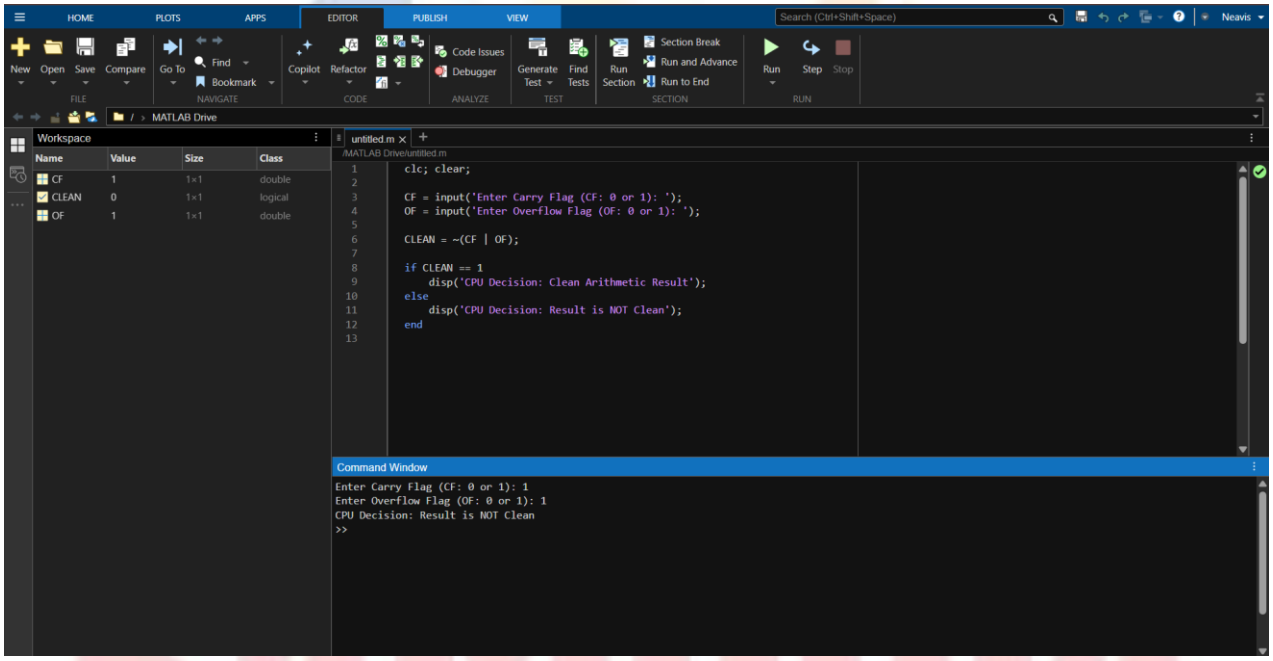
Input:

Enter Carry Flag (CF: 0 or 1): 1

Enter Overflow Flag (OF: 0 or 1): 1

Output:

CPU Decision: Abnormal Result Detected



XIV. Observations and Dimensional Measurement

S. No.	Test Case ID	Input Value A	Input Value B	Operation Performed	ALU Status Flags (CF, OF, SF, PF)	Logic Expression Applied	Logic Output (0/1)	CPU Decision / Remarks
1	TC01	10	20	A + B	(0, 0, 0, 1)	CLEAN = ~(CF | OF)	1	Clean Arithmetic Result
2	TC02	127	1	A + B	(0, 1, 1, 0)	ABNORMAL = CF | OF	1	Abnormal Result Detected (Overflow)

3	TC03	-50	-20	A + B	(1, 0, 1, 1)	ABNORMAL = CF | OF	1	Abnormal Result (Carry Generated)
4	TC04	15	-40	A + B	(0, 0, 1, 0)	VALID_NEGATIVE = NAND($\overline{OF}$, SF)	0	Valid Negative Result

Note: Instructions for Filling the Table:

- Test Case ID:
  - Assign a unique identifier (Case–1, Case–2, etc.)
- ALU Status Flags:
  - Record CF, OF, SF, PF obtained from MATLAB program.
- Logic Expression Applied:
  - Example: C$\overline{F}$ AND O$\overline{F}$ AND PF or NAND(S$\overline{F}$, OF)
- Logic Output:
  - Record 1 for TRUE, 0 for FALSE.
- CPU Decision / Remarks:
  - Example: Safe output, Valid negative result, Rejected due to overflow.

XV. Results/Observations

1. ALU status flags were generated in MATLAB and logic gates were implemented using logical operators.
2. Different conditions such as safe, valid negative, abnormal, and clean results were correctly detected using logic expressions.
3. The observed logic outputs matched the expected CPU decisions for all test cases.

XVI. Interpretation of Results

1. The obtained results show that ALU status flags can be effectively used to determine the correctness of arithmetic operations.
2. Logical gate–based expressions help the CPU distinguish between safe, clean, valid negative, and abnormal results.
3. The experiment confirms that combining hardware flags with logic operations improves reliable decision-making in CPU operations.

XVII. Conclusions

1. This experiment successfully demonstrated the use of ALU status flags and logic gates to evaluate arithmetic operation results. MATLAB was effectively used to simulate CPU decision-making for safe, clean, valid negative, and abnormal conditions.
2. The observed results matched theoretical expectations, confirming the correctness of the logic-based approach.

XVIII. Suggested Practicum Related Questions:

Note: *Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.*

1. ***Justify*** how XOR(SF, OF) can be used to detect signed overflow and justify its correctness..
2. ***Determine*** how XNOR(SF, OF) confirms a valid signed result.
3. ***Recommend*** a logic expression using OR(CF, OF) to identify any arithmetic error.
4. ***Generate*** a condition using NOR(CF, OF) to detect a completely clean arithmetic result.
5. ***Justify*** why NAND is preferred over AND in certain CPU control circuits.

XIX References / Suggestions for further reading

1. Computer Organization and Architecture – ALU and Status Flags

2. Digital Logic Design – Logic Gates and Applications
3. Processor Architecture Textbooks

XX Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the *'process-related skills/LOs'* (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and *'product-related skills/LOs'*

Note: The weightage for the ***process-related skills*** and ***product-related skills*** will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators		Maximum Marks	Marks Obtained
Process-related Skills: 18 Marks - 60%			
1	Correctly creating and running MATLAB scripts (coding hygiene, commenting, workspace handling).	6	
2	Selecting appropriate arithmetic operations for the given data.	5	
3	Following safety, coding ethics, and good laboratory practices (file management, backups).	4	
4	Contribution as a team member (peer collaboration, sharing datasets, debugging).	3	
Product-related Skills: 12 Marks - 40%			
1	Producing accurate computation results and well-formatted MATLAB Table output.	3	
2	Correct interpretation of logical comparisons.	2	
3	Interpretation of results (clarity, mathematical validity).	3	
4	Drawing meaningful conclusions using NEP-aligned verbs (justify, determine, generate).	2	
5	Ability to answer practicum-related higher-order questions.	1	
6	Submitting the report on time with screenshots and code	1	
	Total	**30**	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	**NEAVIS RISHVA. J**	**URK25CS1236**

To be filled by the Faculty Member:

Marks Obtained			Name and Designation of the Faculty Member	Date of Evaluation
Process-Related Skills (18)	**Product-Related Skills (12)**	**Total (30)**		

****